

Sommaire

ERO1 — Déneigement de Montréal : explication complète du projet	1
1. Le problème en une phrase	1
2. Tous les paramètres de départ	2
Paramètres économiques (municipaux) — src/cost.py	2
Paramètres de modélisation	2
Hypothèses simplificatrices (à connaître, le coach demande souvent les limites)	2
3. Le pipeline complet (ordre d'exécution, demo.py)	2
4. Chaque algorithme expliqué	2
Algo 0 — Extraction des données (io_osm.py)	2
Algo 1 — Plus grande composante fortement connexe (largest_strongly_connected)	3
Algo 2 — Classification des priorités (priorities.py)	3
Algo 3 — Détection des services essentiels (annotate_essential + _nearest_nodes)	3
Algo 4 — Postier chinois dirigé (directed_cpp) — LE cœur	3
Algo 5 — Postier rural dirigé (rural_postman) — pour S1 et S2	4
Algo 6 — Construction des 3 scénarios (scenarios.py)	4
Algo 7 — Simulation de remise en service (simulate_clearing)	4
Algo 8 — Modèle de coût (cost.vehicle_cost)	4
Algo 9 — Dimensionnement de la flotte (fleet.py)	4
Algo 10 — Recommandation de flotte (recommend_fleet_size)	5
5. Résultats clés à citer	5
Sur Anjou (220,6 km, 1 véhicule de référence)	5
Dimensionnement des 4 secteurs (S3, remise ≤ 8 h)	5
6. Matrice éthique (analyse d'impact)	5
7. Questions probables du coach (et réponses)	6
8. Fonctionnalités du code non détaillées dans le rapport (mais dans l'app)	6
Accessibilité aux services (accessibility.py) — indicateur d'impact social	6
Tronçons en travaux (roadworks.py) — robustesse / scénario de perturbation	6
Orchestration (simulate.py)	6
Export (exports.py)	6
Visualisation (figures.py, viz.py) — pas des algos	6
Récapitulatif des 3 « briques » d'algo RO du cours utilisées	7

ERO1 — Déneigement de Montréal : explication complète du projet

Fiche de préparation au point coach. Couvre : le problème, tous les paramètres de départ, chaque algorithme dans l'ordre du pipeline, les résultats, et les questions probables.

1. Le problème en une phrase

On doit faire passer une déneigeuse sur **chaque rue** de 4 arrondissements de Montréal (Outremont, Verdun, Anjou, Rivière-des-Prairies-Pointe-aux-Trembles) **au moins une fois**, en minimisant la distance totale, tout en respectant les sens uniques. C'est le **problème du postier chinois dirigé** (Directed Chinese Postman Problem, DCPD).

Le réseau est modélisé comme un **graphe orienté** $G = (V, A)$: - V = les intersections (nœuds), - A = les tronçons de rue (arcs orientés), - chaque arc a porte une longueur $w_a > 0$ (en mètres).

Pourquoi un graphe **orienté** et pas un TSP ? On couvre des **arcs** (les rues), pas des sommets. Le TSP visite des sommets et est NP-difficile ; le postier chinois dirigé est **résoluble exactement en temps polynomial**. C'est le cœur de la justification du choix de méthode.

2. Tous les paramètres de départ

Paramètres économiques (municipaux) — `src/cost.py`

Paramètre	Valeur	Variable
Coût fixe par véhicule/jour	500 \$	FIXED_COST
Coût kilométrique	1,1 \$/km	KM_COST
Coût horaire ≤ 8 h	1,1 \$/h	HOURLY_COST_NORMAL
Coût horaire > 8 h	1,3 \$/h	HOURLY_COST_OVERTIME
Vitesse moyenne	10 km/h	SPEED_KMH
Seuil heures sup.	8 h	NORMAL_HOURS

Paramètres de modélisation

- **Type de réseau OSM** : `network_type="drive"` (réseau carrossable, respecte les sens uniques).
- **Rayon « service essentiel »** : `radius_m = 120` m autour d'un point d'intérêt (`priorities.py`).
- **Borne de temps cible flotte** : `max_hours = 8` h (`recommend_fleet_size`).
- **k_max** (nombre max de véhicules testés) : 15 dans `demo.py`.
- **Classes de priorité** déduites du type highway OSM (table ci-dessous).

Hypothèses simplificatrices (à connaître, le coach demande souvent les limites)

- Une rue est déneigée **dès le 1er passage** ; les passages suivants sont des déplacements à **vide** (*deadhead*).
- **Vitesse constante** 10 km/h → la durée est proportionnelle à la distance.
- Toutes les déneigeuses sont **identiques** et partent/reviennent d'un **dépôt unique**.
- On travaille sur la **plus grande composante fortement connexe** de chaque secteur (garantit qu'un circuit existe).
- Hors périmètre : épandage de sel, chargement/évacuation de la neige, congestion variable, largeur des voies. Une déneigeuse = une voie par passage.

3. Le pipeline complet (ordre d'exécution, `demo.py`)

1. Charger le réseau OSM → `io_osm.load_sector`
2. Annoter les priorités (1/2/3) → `priorities.annotate_priorities`
3. Charger les POI sensibles → `io_osm.load_pois`
4. Marquer les rues "essentiels" → `priorities.annotate_essential`
5. Évaluer les 3 scénarios S1/S2/S3 → `scenarios.evaluate_scenario`
 - └ construire la tournée → `build_route (rural_postman / directed_cpp)`
 - └ simuler la remise en svc → `simulate_clearing`
 - └ calculer le coût → `cost.vehicle_cost`
6. Dimensionner la flotte sur S3 → `fleet.cost_vs_vehicles + recommend_fleet_size`
7. Écrire résultats + cartes + figs

4. Chaque algorithme expliqué

Algo 0 — Extraction des données (`io_osm.py`)

On télécharge via **osmnx** le graphe routier de chaque arrondissement (`ox.graph_from_place(..., network_type="drive")`) et les points d'intérêt sensibles (`ox.features_from_place` avec les tags : hôpital, clinique, école, maternelle, caserne, police, université, arrêt de bus, station de métro/tram). Tout est mis en cache sur disque (`data/*.graphml, *.gpkg`) pour fonctionner **hors ligne**.

Algo 1 — Plus grande composante fortement connexe (largest_strongly_connected)

On garde le plus gros sous-graphe où **tout nœud peut atteindre tout autre nœud** en respectant les sens uniques. - **Pourquoi** : sans forte connexité, certains nœuds en déséquilibre ne peuvent pas être reliés → **aucun circuit eulérien possible**. C'est la condition d'existence de la solution. - Couvre en pratique la quasi-totalité de la voirie.

Algo 2 — Classification des priorités (priorities.py)

Chaque rue reçoit une **classe** d'après son type highway OSM :

Classe	Signification	Types OSM
1	Axes structurants	motorway, trunk, primary (+ links)
2	Voies collectrices	secondary, tertiary, busway
3	Desserte locale	residential, living_street, service, unclassified...

Par défaut, une rue non reconnue → classe 3.

Algo 3 — Détection des services essentiels (annotate_essential + _nearest_nodes)

Pour chaque POI sensible, on cherche le **nœud du réseau le plus proche** : - distance euclidienne approchée en degrés, **corrigée par le cosinus de la latitude** (suffisant à l'échelle d'un arrondissement, évite la dépendance à scikit-learn) ; - on prend l'argmin de $dx^2 + dy^2$ sur tous les nœuds. - Tout arc incident à un nœud « essentiel » est marqué `essential = True`. - **Repli robuste** : si les POI sont indisponibles (panne réseau), on considère comme essentielles les rues de classe 1 et 2 (les services sont en pratique sur/près du réseau structurant).

Algo 4 — Postier chinois dirigé (directed_cpp) — LE cœur

C'est la méthode centrale. Trois étapes :

4a. Calcul du déséquilibre (node_imbalance) Un graphe orienté admet un **circuit eulérien** (qui passe une et une seule fois par chaque arc) **ssi en chaque nœud : degré entrant = degré sortant**. On mesure l'écart :

$$\text{imbalance}(v) = \text{degré_sortant}(v) - \text{degré_entrant}(v)$$

Si tous les imbalance valent 0, le graphe est déjà eulérien (rien à ajouter).

4b. Rééquilibrage par FLOT À COÛT MINIMUM (_balance_by_min_cost_flow) — le point clé du cours Pour rééquilibrer, il faut **ajouter le minimum de passages supplémentaires** (arcs dupliqués) le long des rues existantes. Trouver ces duplications au coût minimal est **exactement un problème de flot à coût minimum** : - on construit un réseau de flot où chaque nœud a une `demand = imbalance(v)` ; - les arcs ont pour **coût** la longueur de la rue la moins chère entre u et v (`_cheapest_parallel_arc`), et une **capacité infinie** (on peut repasser autant de fois que nécessaire) ; - on appelle `nx.min_cost_flow(F)`.

On « expédie » des passages des nœuds en **excès de sortie** vers les nœuds en **excès d'entrée**, au coût de la longueur des rues empruntées. Comme $\sum \text{imbalance} = 0$ et que les déséquilibres sont entiers, la **solution est entière** (pas besoin d'arrondir). Le résultat : un dict $(u,v) \rightarrow$ nombre de passages à ajouter.

À retenir pour le coach : « rééquilibrer un graphe orienté pour le rendre eulérien au coût minimal = un flot à coût minimum ». C'est ce qui rend la méthode **exacte** et **polynomiale**.

4c. Multigraphe eulérien + circuit (build_eulerian_multigraph + Hierholzer) On construit un multigraphe H = chaque rue une fois (`duplicate=False`) + les duplications du flot (`duplicate=True`).

Ce H est eulérien par construction. On en extrait le circuit avec l'**algorithme de Hierholzer** (`nx.eulerian_circuit`), qui produit l'ordre des arcs à parcourir.

Sorties (stats) : `required_m` (distance utile), `deadhead_m` (distance à vide), `total_m`, `deadhead_ratio` (% à vide), etc.

Algo 5 — Postier rural dirigé (`rural_postman`) — pour S1 et S2

Quand on ne couvre qu'un **sous-ensemble** de rues (ex. seulement les axes), c'est le **postier rural dirigé**. Même mécanique, en 3 temps : 1. **Connecter** les composantes du sous-graphe requis entre elles par des **plus courts chemins** (Dijkstra, `nx.shortest_path` pondéré par `length`), ajoutés en déplacements à vide ; 2. **Rééquilibrer** les degrés par flot à coût minimum (même algo qu'en 4b) ; 3. **Extraire** un circuit eulérien (Hierholzer), avec un repli sur la composante du nœud de départ si besoin.

Algo 6 — Construction des 3 scénarios (`scenarios.py`)

Les 3 scénarios traitent **toute** la voirie ; ils diffèrent par l'**ordre** :

Scénario	Ordre des « passes »	Méthode
S1 — Axes d'abord	classe 1, puis 2, puis 3	postier rural × 3 passes
S2 — Services essentiels	essentiels, puis le reste	postier rural × 2 passes
S3 — Coût minimal	aucune priorité, tout d'un coup	postier chinois pur (optimum de distance)

`build_route` enchaîne les passes : entre deux passes, le véhicule rejoint le départ de la passe suivante par un plus court chemin (à vide), et termine par un retour au dépôt.

Algo 7 — Simulation de remise en service (`simulate_clearing`)

On parcourt la tournée en cumulant la distance ; pour chaque rue on note l'instant de **premier passage** (= moment où elle est déneigée). Puis, par classe et pour les essentiels, on prend le **max** des premiers passages = instant où la **dernière** rue de cette catégorie est dégagée. Converti en heures via / `SPEED_KMH`. → C'est l'indicateur « **temps de remise en service** » (axes, essentiels, fin totale).

Algo 8 — Modèle de coût (`cost.vehicle_cost`)

Pour une distance donnée :

```

durée = distance / 10
heures_normales = min(durée, 8)
heures_sup = max(0, durée - 8)
coût = 500 (fixe) + 1,1 × km + (1,1 × h_normales + 1,3 × h_sup)

```

Algo 9 — Dimensionnement de la flotte (`fleet.py`)

Heuristique « **route d'abord, découpe ensuite** » (route-first, split-second) : 1. calculer la tournée d'un véhicule (scénario S3) ; 2. la découper en k tronçons contigus de longueur ≈ égale ; 3. chaque véhicule : dépôt → son tronçon → dépôt (les liaisons sont des plus courts chemins à vide).

Affinement multi-passes — Algorithme Hongrois (`linear_sum_assignment`, `scipy`) : quand il y a plusieurs passes, on affecte **optimalement** les k véhicules aux k tronçons pour **minimiser le déplacement à vide** total. La matrice de coût = distances (`Dijkstra single_source_dijkstra_path_length`) entre la position courante de chaque véhicule et le début de chaque tronçon ; l'algorithme hongrois résout l'affectation optimale.

Effet : les k véhicules travaillent en parallèle → le **temps** de remise en service décroît en $\sim 1/k$, tandis que le **coût** total croît (dominé par le coût fixe de 500 \$/véhicule). C'est le compromis « coût = $f(\text{nombre de véhicules})$ ».

Algo 10 — Recommandation de flotte (recommend_fleet_size)

On teste $k = 1..15$, et on retient le **plus petit k** dont le temps de remise en service ≤ 8 h. À défaut, celui de temps minimal.

5. Résultats clés à citer

Sur Anjou (220,6 km, 1 véhicule de référence)

Scénario	Dist. (km)	À vide	Coût/véh	Axes dégagés	Essentiels	Fin
S1 — Axes	362	64 %	944 \$	3,6 h	35,6 h	35,7 h
S2 — Essentiels	400	81 %	990 \$	39,8 h	16,4 h	39,8 h
S3 — Coût min	265	20 %	824 \$	26,4 h	26,4 h	26,4 h

Lecture : aucun scénario ne domine. S3 = le moins cher mais aveugle au service (tout à 26 h). S1 dégage les axes **10× plus vite** (3,6 h) mais +15 % de coût et 64 % à vide. S2 dégage les essentiels en 16 h mais c'est le plus cher (services dispersés → 81 % à vide).

Dimensionnement des 4 secteurs (S3, remise ≤ 8 h)

Secteur	Voirie (km)	Tournée S3 (km)	Flotte	Coût/jour
Outremont	70	81	2	1 103 \$
Verdun	102	110	2	1 145 \$
Anjou	221	265	4	2 353 \$
RDP-PAT	707	786	13	7 634 \$
Total	1 100	1 242	21	12 235 \$

Pour Anjou : le temps passe **sous 8 h à partir de 4 véhicules** (7,6 h, 2 353 \$/jour).

6. Matrice éthique (analyse d'impact)

Méthode de **Mepham** : on croise les parties prenantes $\times$ 3 valeurs — **bien-être** (utilitarisme), **autonomie** (déontologie), **justice** (équité). - **S1** sert les automobilistes / l'économie, mais désavantage les quartiers résidentiels (équité spatiale). - **S2** corrige une inégalité d'accès (vulnérables, secours) mais coûte cher et retarde la majorité. - **S3** traite tout le monde « pareil » et au moindre coût, mais sans aucune garantie de service.

Recommandation finale : une politique **hybride** — essentiels + axes en 1re vague, puis desserte locale au coût minimal — combine l'essentiel des bénéfices.

7. Questions probables du coach (et réponses)

- **Pourquoi pas un TSP ?** On couvre des arcs, pas des sommets ; le TSP est NP-difficile, le postier chinois dirigé est exact et polynomial.
 - **Pourquoi pas Dijkstra seul ?** Dijkstra relie deux points, il ne couvre pas toutes les rues.
 - **Votre méthode est-elle optimale ?** Oui pour S3 (postier chinois exact = flot à coût min + Euler). Pour S1/S2 (rural) et pour le **découpage de flotte**, c'est **heuristique** → borne supérieure sur le coût. La forte part de déplacements à vide en S1/S2 vient de là ; un solveur de tournées plus fin la réduirait.
 - **Comment garantisiez-vous qu'un circuit existe ?** Forte connexité + rééquilibrage des degrés par flot → multigraphe eulérien.
 - **Limites ?** Vitesse constante (ignore congestion/météo), dépôt unique, déneigement instantané (pas de file de chargement), découpe de flotte non optimale. Acceptable pour **comparer des scénarios** sur une journée type.
-

8. Fonctionnalités du code non détaillées dans le rapport (mais dans l'app)

Ces modules existent dans `src/` et sont utilisés par l'application interactive (`simulate.run_simulation` + `Streamlit`). Bon à connaître si le coach explore le code.

Accessibilité aux services (`accessibility.py`) — indicateur d'impact social

Répond à : « à un instant donné, quelle part de la population peut rejoindre un service essentiel en n'empruntant que des rues déjà déneigées ? » - **Proxy de population** (`node_population_weight`) : faute de données de recensement, chaque intersection est pondérée par la longueur de **desserte locale (classe 3)** incidente — là où les gens habitent. - **Calcul** (`accessibility_curve`) : à 12 instants de la tournée, on construit le sous-réseau des rues **déjà déneigées**, puis par un **BFS sur le graphe inversé** depuis les nœuds « services essentiels » (`_reverse_reachable`), on trouve qui peut les atteindre. On somme les poids population et on divise par le total → un **% d'accessibilité**. - **Sorties** : courbe (temps, %), accessibilité finale, et **temps pour atteindre 50 % et 90 % d'accessibilité**. C'est l'indicateur qui rend S2 « lisible » socialement.

Tronçons en travaux (`roadworks.py`) — robustesse / scénario de perturbation

`apply_roadworks(G, n_blocked, seed)` ferme aléatoirement `n_blocked` rues (les deux sens), puis reprend la **plus grande composante fortement connexe**. Les déneigeuses contournent les fermetures ; le coût et l'accessibilité reflètent l'impact. Permet de tester la résilience du plan.

Orchestration (`simulate.py`)

`run_simulation(...)` = point d'entrée unique de l'app : applique les travaux → évalue le scénario → découpe la flotte → calcule coût + accessibilité → renvoie tout pour l'affichage. Paramètres : `scenario, n_vehicles, n_blocked, seed, speed_kmh, k_max=15`.

Export (`exports.py`)

Génère une archive ZIP : trace **GPX horodatée** par déneigeuse (départ conventionnel **06:00, 15 janvier 2026**), **GeoJSON** des tournées, et stats **CSV/JSON**.

Visualisation (`figures.py, viz.py`) — pas des algos

- `figures.py` : PNG du rapport (matplotlib, backend Agg) — carte par priorité, courbe coût, compromis scénarios.
- `viz.py` : cartes interactives **folium/Leaflet** — réseau coloré, services en clusters, tournées **animées** (icône flocon par véhicule), travaux en rouge hachuré.

Récapitulatif des 3 « briques » d'algo RO du cours utilisées

1. **Flot à coût minimum** → rééquilibrage des degrés (cœur du postier chinois).
2. **Plus courts chemins (Dijkstra)** → connexion des composantes + liaisons à vide.
3. **Affectation optimale (algorithme hongrois)** → répartition flotte/passes.

Plus : circuit eulérien (**Hierholzer**), composantes **fortement connexes**.